剪刀石头布

剪刀石头布
例程实验效果
代码讲解
代码详细结构
关键代码

例程实验效果

本节我们来学习一下基于K230的手势识别，做一个剪刀石头布的游戏

游戏执行过程如下：

1. 将例程代码复制到CanMV IDE中，点击左下角运行按键。请确保这个时候摄像头中没有“手”出现
2. 等待左下角出现”Total took xxxxx“代表游戏开始

```
find sensor gc2093_csi2, type 23, output 1920x1080@30
buffer pool :  5
sensor(0), mode 0, buffer_num 4, buffer_size 0
total took 57.00 ms
total took 58.00 ms
total took 57.00 ms
total took 55.00 ms
total took 57.00 ms
total took 55.00 ms
```

3. 游戏开始后，我们用手做出“石头”、“剪刀”或者布的形状，出现在镜头里。这个时候屏幕上会给出程序的出拳，并得出输赢结果。得到结过后，代表这一回合结束，请把手移除摄像头的范围。



4. 游戏总共进行三回合，三回合后，程序会根据三局两胜的规则得出最终的结果。【这里有一个小BUG存在，如果第三回合结束后程序没有马上给出最终结果，你可以让手再次进入镜头中，程序就会给出最终结果了】



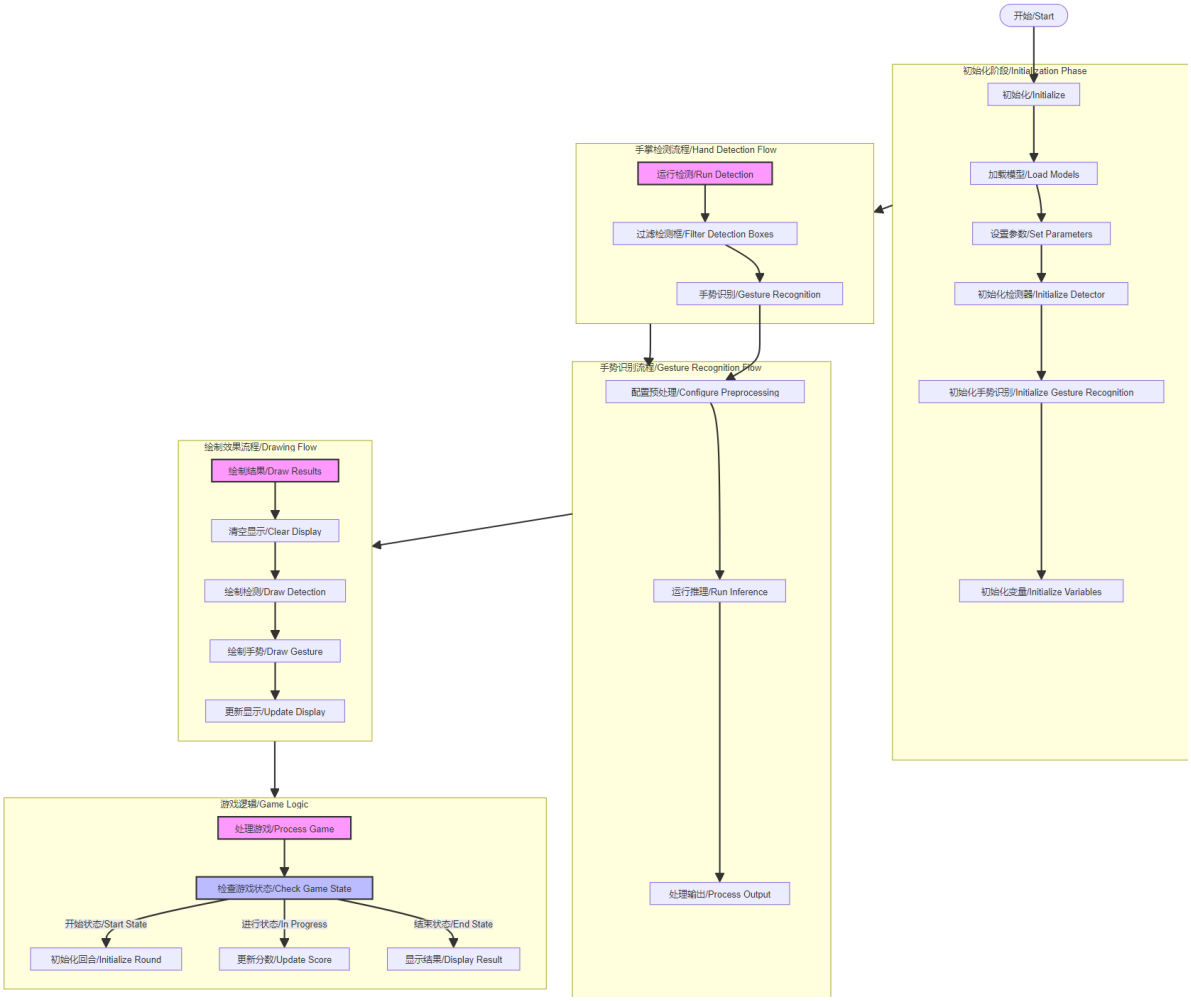
代码讲解

这是一个基于手势识别的猜拳游戏程序。主要包含以下几个部分:

1. 手掌检测类 (HandDetApp) 和手势关键点分类类 (HandKPClassApp)
    - 这两个类分别实现了手掌检测和手势关键点识别的功能,为猜拳游戏提供底层支持。
    - 它们封装了图像预处理、模型推理和后处理等操作,为上层逻辑提供统一的接口。
2. 猜拳游戏类 (FingerGuess)
    - 这个类负责整个猜拳游戏的执行逻辑。
    - 它会首先调用手掌检测和关键点分类模型,获取用户当前的手势。
    - 然后根据不同的游戏模式(必赢、必输、多局),生成对应的电脑出拳,并判断胜负。
    - 最后将游戏过程和结果绘制到屏幕上显示。
3. 游戏执行函数(exce_demo)
    - 这个函数负责初始化游戏相关的环境和资源,并进入游戏的主循环。
    - 在主循环中,它不断获取当前帧图像,调用FingerGuess类的run方法处理图像,最后显示游戏画面。

整个游戏的执行流程是:

1. 初始化游戏环境和资源
2. 进入游戏主循环
3. 在每一帧中:
    - 获取当前图像
    - 检测手掌,识别手势
    - 根据游戏逻辑生成电脑出拳,判断胜负
    - 绘制游戏画面并显示



代码详细结构

1. 初始化阶段/Initialization Phase:
    - 加载检测和识别模型/Load detection and recognition models
    - 设置各种参数/Set parameters
    - 初始化检测器和手势识别模块/Initialize detector and gesture recognition
    - 初始化游戏相关变量/Initialize game-related variables
2. 手掌检测流程/Hand Detection Flow:
    - 运行手掌检测/Run hand detection
    - 过滤检测结果/Filter detection boxes
    - 传递给手势识别模块/Pass to gesture recognition
3. 手势识别流程/Gesture Recognition Flow:
    - 配置预处理流程/Configure preprocessing
    - 执行模型推理/Run model inference
    - 处理推理输出/Process output
4. 绘制效果流程/Drawing Flow:
    - 清空显示/Clear display
    - 绘制检测结果/Draw detection
    - 绘制手势识别结果/Draw gesture recognition
    - 更新显示/Update display
5. 游戏逻辑/Game Logic:
    - 处理游戏逻辑/Process game logic
    - 检查游戏状态/Check game state

- 初始化新回合/Initialize new round
- 更新玩家和AI的得分/Update player and AI scores
- 显示最终结果/Display final result

关键代码

```
# 猜拳游戏任务类 / Rock-paper-scissors game task class
class FingerGuess:
    def __init__(self, hand_det_kmodel, hand_kp_kmodel, det_input_size,
kp_input_size, labels, anchors, confidence_threshold=0.25, nms_threshold=0.3,
nms_option=False, strides=[8, 16, 32], guess_mode=3, rgb888p_size=[1280, 720],
display_size=[1920, 1080], debug_mode=0):
        # 手掌检测模型路径 / Hand detection model path
        self.hand_det_kmodel = hand_det_kmodel
        # 手掌关键点模型路径 / Hand keypoint model path
        self.hand_kp_kmodel = hand_kp_kmodel
        # 手掌检测模型输入分辨率 / Hand detection model input resolution
        self.det_input_size = det_input_size
        # 手掌关键点模型输入分辨率 / Hand keypoint model input resolution
        self.kp_input_size = kp_input_size
        self.labels = labels
        # anchors / Anchor boxes
        self.anchors = anchors
        # 置信度阈值 / Confidence threshold
        self.confidence_threshold = confidence_threshold
        # nms阈值 / Non-Maximum Suppression threshold
        self.nms_threshold = nms_threshold
        # nms选项 / NMS option
        self.nms_option = nms_option
        # 特征图针对输入的下采样倍数 / Feature map downsampling multipliers relative
to input
        self.strides = strides
        # sensor给到AI的图像分辨率，宽16字节对齐 / Image resolution from sensor to
AI, width aligned to 16 bytes
        self.rgb888p_size = [ALIGN_UP(rgb888p_size[0], 16), rgb888p_size[1]]
        # 视频输出VO分辨率，宽16字节对齐 / Video output VO resolution, width aligned
to 16 bytes
        self.display_size = [ALIGN_UP(display_size[0], 16), display_size[1]]
        # debug_mode模式 / Debug mode
        self.debug_mode = debug_mode
        self.guess_mode = guess_mode
        # 石头剪刀布的贴图array / Rock-paper-scissors image arrays
        self.five_image = self.read_file("/sdcard/utils/five.bin")
        self.fist_image = self.read_file("/sdcard/utils/fist.bin")
        self.shear_image = self.read_file("/sdcard/utils/shear.bin")
        self.counts_guess = -1
            # 猜拳次数 计数 / Count of rock-paper-scissors rounds
        self.player_win = 0
             # 玩家 赢次计数 / Player win count
        self.k230_win = 0
             # k230 赢次计数 / K230 win count
        self.sleep_end = False
            # 是否 停顿 / Whether to pause
        self.set_stop_id = True
             # 是否 暂停猜拳 / Whether to suspend the game
```

```
        self.LIBRARY = ["fist", "yeah", "five"]
             # 猜拳 石头剪刀布 三种方案的dict / Dictionary of three rock-paper-
scissors options
        self.hand_det = HandDetApp(self.hand_det_kmodel, self.labels,
model_input_size=self.det_input_size, anchors=self.anchors,
confidence_threshold=self.confidence_threshold,
nms_threshold=self.nms_threshold, nms_option=self.nms_option,
strides=self.strides, rgb888p_size=self.rgb888p_size,
display_size=self.display_size, debug_mode=0)
        self.hand_kp = HandKPClassApp(self.hand_kp_kmodel,
model_input_size=self.kp_input_size, rgb888p_size=self.rgb888p_size,
display_size=self.display_size)
        self.hand_det.config_preprocess()

    # run函数 / Run function
    def run(self, input_np):
        # 先进行手掌检测 / First perform hand detection
        det_boxes = self.hand_det.run(input_np)
        boxes = []
        gesture_res = []
        for det_box in det_boxes:
            # 对检测的手做手势识别 / Perform gesture recognition on detected hands
            x1, y1, x2, y2 = det_box[2], det_box[3], det_box[4], det_box[5]
            w, h = int(x2 - x1), int(y2 - y1)
            if (h < (0.1 * self.rgb888p_size[1])):
                continue
            if (w < (0.25 * self.rgb888p_size[0]) and ((x1 < (0.03 *
self.rgb888p_size[0])) or (x2 > (0.97 * self.rgb888p_size[0])))):
                continue
            if (w < (0.15 * self.rgb888p_size[0]) and ((x1 < (0.01 *
self.rgb888p_size[0])) or (x2 > (0.99 * self.rgb888p_size[0])))):
                continue
            self.hand_kp.config_preprocess(det_box)
            results_show, gesture = self.hand_kp.run(input_np)
            boxes.append(det_box)
            gesture_res.append(gesture)
        return boxes, gesture_res

    # 绘制效果 / Draw results
    def draw_result(self, pl, dets, gesture_res):
        pl.osd_img.clear()
        # 手掌的手势分类得到用户的出拳，根据不同模式给出开发板的出拳，并将对应的贴图放到屏幕上
显示
        # Get user's gesture from hand gesture classification, determine the
board's gesture based on different modes, and display corresponding images on
screen
        if (len(dets) >= 2):
            pl.osd_img.draw_string_advanced(self.display_size[0] // 2 - 50,
self.display_size[1] // 2 - 50, 30, "请保证只有一只手入镜 Make sure only one hand on
the screen", color=(255, 255, 0, 0))
        elif (self.guess_mode == 0):
            draw_img_np = np.zeros((self.display_size[1], self.display_size[0],
4), dtype=np.uint8)
            draw_img = image.Image(self.display_size[0], self.display_size[1],
image.ARGB8888, alloc=image.ALLOC_REF, data=draw_img_np)
            if (gesture_res[0] == "fist"):
                draw_img_np[:400, :400, :] = self.shear_image
            elif (gesture_res[0] == "five"):
```

```
                draw_img_np[:400, :400, :] = self.fist_image
            elif (gesture_res[0] == "yeah"):
                draw_img_np[:400, :400, :] = self.five_image
            pl.osd_img.copy_from(draw_img)
        elif (self.guess_mode == 1):
            draw_img_np = np.zeros((self.display_size[1], self.display_size[0],
4), dtype=np.uint8)
            draw_img = image.Image(self.display_size[0], self.display_size[1],
image.ARGB8888, alloc=image.ALLOC_REF, data=draw_img_np)
            if (gesture_res[0] == "fist"):
                draw_img_np[:400, :400, :] = self.five_image
            elif (gesture_res[0] == "five"):
                draw_img_np[:400, :400, :] = self.shear_image
            elif (gesture_res[0] == "yeah"):
                draw_img_np[:400, :400, :] = self.fist_image
            pl.osd_img.copy_from(draw_img)
        else:
            draw_img_np = np.zeros((self.display_size[1], self.display_size[0],
4), dtype=np.uint8)
            draw_img = image.Image(self.display_size[0], self.display_size[1],
image.ARGB8888, alloc=image.ALLOC_REF, data=draw_img_np)
            if (self.sleep_end):
                time.sleep_ms(2000)
                self.sleep_end = False
            if (len(dets) == 0):
                self.set_stop_id = True
                return
            if (self.counts_guess == -1 and gesture_res[0] != "fist" and
gesture_res[0] != "yeah" and gesture_res[0] != "five"):
                draw_img.draw_string_advanced(self.display_size[0] // 2 - 50,
self.display_size[1] // 2 - 50, 30, "游戏开始 GAME START", color=(255, 255, 0, 0))
                time.sleep_ms(500)
            elif (self.counts_guess == self.guess_mode):
                draw_img.clear()
                if (self.k230_win > self.player_win):
                    draw_img.draw_string_advanced(self.display_size[0] // 2 -
50, self.display_size[1] // 2 - 50, 30, "你输了 You lose", color=(255, 255, 0,
0))
                elif (self.k230_win < self.player_win):
                    draw_img.draw_string_advanced(self.display_size[0] // 2 -
50, self.display_size[1] // 2 - 50, 30, "你赢了 You win", color=(255, 255, 0, 0))
                else:
                    draw_img.draw_string_advanced(self.display_size[0] // 2 -
50, self.display_size[1] // 2 - 50, 30, "平局 | tie", color=(255, 255, 0, 0))
                self.counts_guess = -1
                self.player_win = 0
                self.k230_win = 0
                self.sleep_end = True
            else:
                if (self.set_stop_id):
                    if (self.counts_guess == -1 and (gesture_res[0] == "fist" or
gesture_res[0] == "yeah" or gesture_res[0] == "five")):
                        self.counts_guess = 0
                    if (self.counts_guess != -1 and (gesture_res[0] == "fist" or
gesture_res[0] == "yeah" or gesture_res[0] == "five")):
                        k230_guess = randint(1, 10000) % 3
                        if (gesture_res[0] == "fist" and
self.LIBRARY[k230_guess] == "yeah"):
```

```
                            self.player_win += 1
                        elif (gesture_res[0] == "fist" and
self.LIBRARY[k230_guess] == "five"):
                            self.k230_win += 1
                        if (gesture_res[0] == "yeah" and
self.LIBRARY[k230_guess] == "fist"):
                            self.k230_win += 1
                        elif (gesture_res[0] == "yeah" and
self.LIBRARY[k230_guess] == "five"):
                            self.player_win += 1
                        if (gesture_res[0] == "five" and
self.LIBRARY[k230_guess] == "fist"):
                            self.player_win += 1
                        elif (gesture_res[0] == "five" and
self.LIBRARY[k230_guess] == "yeah"):
                            self.k230_win += 1
                        if (self.LIBRARY[k230_guess] == "fist"):
                            draw_img_np[:400, :400, :] = self.fist_image
                        elif (self.LIBRARY[k230_guess] == "five"):
                            draw_img_np[:400, :400, :] = self.five_image
                        elif (self.LIBRARY[k230_guess] == "yeah"):
                            draw_img_np[:400, :400, :] = self.shear_image
                        self.counts_guess += 1
                        draw_img.draw_string_advanced(self.display_size[0] // 2
- 50, self.display_size[1] // 2 - 50, 30, "第" + str(self.counts_guess) + "回合 |
Round " +  str(self.counts_guess), color=(255, 255, 0, 0))
                        self.set_stop_id = False
                        self.sleep_end = True
                    else:
                        draw_img.draw_string_advanced(self.display_size[0] // 2
- 50, self.display_size[1] // 2 - 50, 30, "第" + str(self.counts_guess + 1) + "回
合 | Round " +  str(self.counts_guess), color=(255, 255, 0, 0))
            pl.osd_img.copy_from(draw_img)

    # 读取石头剪刀布的bin文件方法 / Method to read rock-paper-scissors bin files
    def read_file(self, file_name):
        image_arr = np.fromfile(file_name, dtype=np.uint8)
        image_arr = image_arr.reshape((400, 400, 4))
        return image_arr
```